文件系统

linux文件系统的基本组成

- 索引节点inode：记录数据在磁盘的位置，作为唯一标识，也占用磁盘空间
- 目录项dentry：记录文件名字，索引节点指针。目录项是由内核维护的一个数据结构，不存放于磁盘，而是在内存中缓存

文件数据如何存储在硬盘

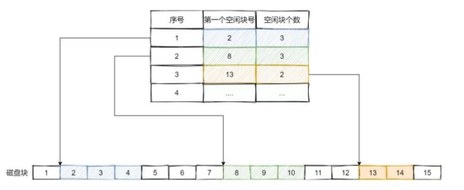
- 磁盘的最小读取单位为扇区，大小为512B
- 磁盘格式化的时候为三个储存区域
  - 超级块：储存文件系统的详细信息
  - 索引节点区：储存索引节点
  - 数据块区：储存文件或目录数据
- 加载到内存的时机
  - 超级块：当文件系统挂载到内存时候
  - 索引节点区：当文件被访问时

虚拟文件系统

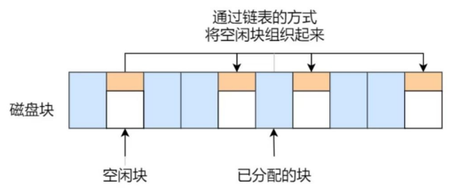
- 操作系统希望对于用户有一个统一的接口，就引入了中间层，这个中间层就是虚拟文件系统
- 文件系统常见有三类
  - 磁盘文件系统：直接把数据存储在磁盘中
  - 内存文件系统：占用内存空间，读写内核的相关程序
  - 网络文件系统：访问其他计算机的文件系统
  - 文件系统要首先挂到某个目录才能正常适用

文件的储存

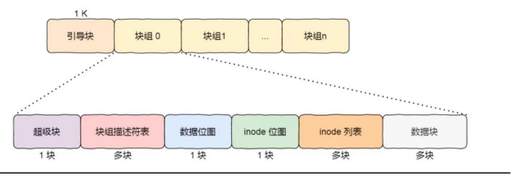
- 储存方式
  - 连续空间存放方式：文件存放在磁盘上连续的物理空间中，文件头需要指定起始块位置和长度，
    - 优点：读写效率高
    - 缺点：磁盘空间缺陷和文件长度不容易扩展
  - 非连续空间存放方式
    - 链表方式：存放数据离散不用连续
      - 优点：消除磁盘碎片，文件长度可以动态库宗翰
      - 缺点：无法直接访问数据块，需要通过指针访问，而指针又浪费掉了一定的内存空间，数据也可能因为软硬件错误导致链表断开数据丢失
    - 索引方式：为每一个文件创建一个索引数据块，里面存放的是指向文件数据块的指针列表，类似于目录
      - 优点
        - 文件的创建增加缩小很方便
        - 没有碎片问题
        - 支持顺序和随机读写
- 管理空闲空间
  - 对于磁盘的空间空间也需要一套合理的方法
  - 方法
    - 空闲表法：为所有空闲空间建立一张表，表的内容包括空闲区的第一块号和该空闲区的块个数，这个方式是连续分配的
      - 优点：对于少量空闲区的时候效果好，适合建立连续文件



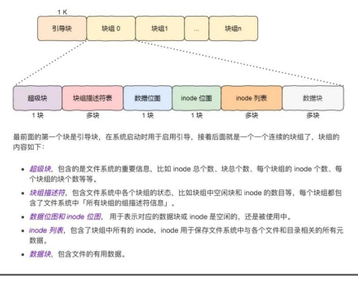
    - 空闲链表法：通过链表的方式管理空闲空间，每一个空闲块有一个指针指向下一个空闲块
      - 优点：简单
      - 缺点：不能随机访问，工作效率低



    - 位图法　　适用二进制的一位来表示一个磁盘中一个盘块的使用情况

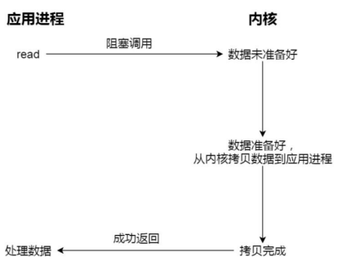


- 文件系统的结构
  - linux适用位图的方法来管理磁盘的使用情况
  - 对于一个块我们还有更大的单位，块组，每一个的内容如下所示

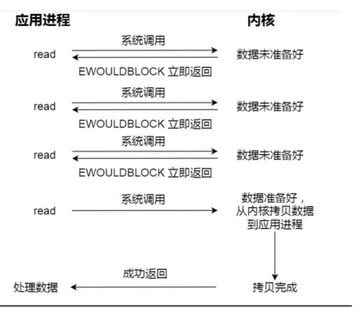

1 K
引导块
块组 0
块组1
...
块组n
超级块
块组描述符表
数据位图
inode 位图
inode 列表
数据块
1 块
多块
1 块
1 块
多块
多块
最前面的第一个块是引导块，在系统启动时用于启用引导，接着后面就是一个一个连续的块组了，块组的内容如下：
- 超级块，包含的是文件系统的重要信息，比如 inode 总个数，块总个数，每个块组的 inode 个数，每个块组的块个数等等。
- 块组描述符，包含文件系统中各个块组的状态，比如块组中空闲块和 inode 的数目等，每个块组都占了文件系统的一块等等的内容是很好理解的。
- 数据位图和 inode 位图，用于表示对应的数据块或 inode 是空闲的，还是被使用中。
- inode 列表，包含了块组中所有的 inode，inode 用于保存文件系统中与各个文件对应这条相关的所有元数据。
- 数据块，包含文件的有用数据。
  - 对于目录，他也是一个特殊的块，普通的块保存的是文件数据，目录的块保存着文件信息，方式是哈希表
  - 有时候我们希望给文件来一个别名，这个别名有两种方式
    - 软连接：重新创建一个文件，它有独立的inode，但是它的文件内容是另外一个文件的路径，软连接是可以跨越文件系统的，有时候目标文件被删除，链接文件还在
    - 硬连接：多个目录项中的索引节点指向同一个文件，指向同一个inode，所以其不可以跨文件系统，只有删除文件的所有硬链接以及源文件的时候，系统才会彻底删除该文件

文件IO

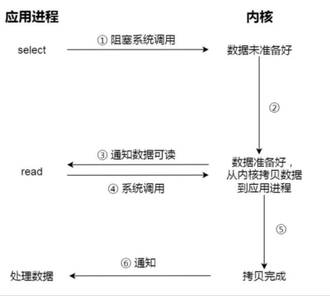
- 缓冲与非缓冲IO（根据是否利用标准库缓冲）
  - 缓冲IO：利用标准库缓存实现文件加速访问，标准库再通过系统调用访问文件
  - 非缓冲IO：直接通过系统调用访问文件，不经过标准库缓存
- 直接与非直接IO（根据是否利用操作系统缓存）
  - 直接IO：不发生内核缓存和用户程序之间的数据复制，直接经过文件系统访问磁盘
  - 非直接IO，读的时候数据从内核缓存拷贝给用户程序，写的时候数据从用户程序拷给内核缓存，再由内核决定什么时候写入数据到磁盘



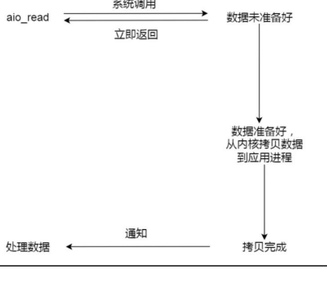
- 阻塞与非阻塞IO与同步与异步IO
  - 阻塞IO：阻塞等待的时内核数据准备好和数据从内核态拷贝到用户态两个过程
  - 非阻塞IO：一直询问直到好了为止



  - 同步IO（IO多路复用）
    - 通过IO事件分发，当内核数据准备好的时候，再以事件通知应用程序进行操作



  - 异步IO（上面三个都是同步IO）
    - 调用一次read后就返回，内核自动将数据从内核空间拷贝到应用程序空间，应用程序不主动发起拷贝操作



相关问题

- 目录项和目录是一个东西吗？不是，目录是文件，在磁盘，目录项是一个数据结构，在内存中
- 如果用了非直接IO进行写数据操作，内核什么情况下才会把缓存数据写入到磁盘
  - 调用write最后发现内核缓存的数据太多，内核会把数据写到磁盘上
  - 用户调用sync，内核缓存会刷新到磁盘上
  - 内存非常紧张无法再分配页面的时候，也会把内核缓存数据刷到磁盘上
  - 内核缓存的数据的缓存时间超过某个时间时，也会把数据刷到磁盘上